# Search For Range

AlgoExpert · Hard · Searching

# The problem

- First and last index of target in a sorted array, [-1, -1] if absent.
- Two modified binary searches.

# Intuition

- Two binary searches: one biased left for the first occurrence, one biased right for the last.

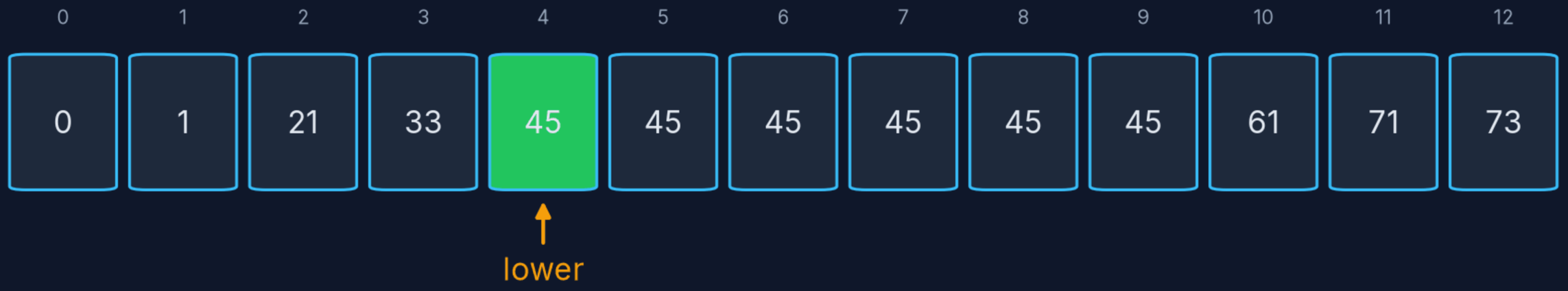
# Brute force vs optimal

- Linear scan → O(n)
- Binary search then expand → O(n) worst
- Two lower-bound searches → O(log n)

# Approach

- 1. Search keeping left on matches
- 2. Search keeping right on matches
- 3. Return both indices or minus ones

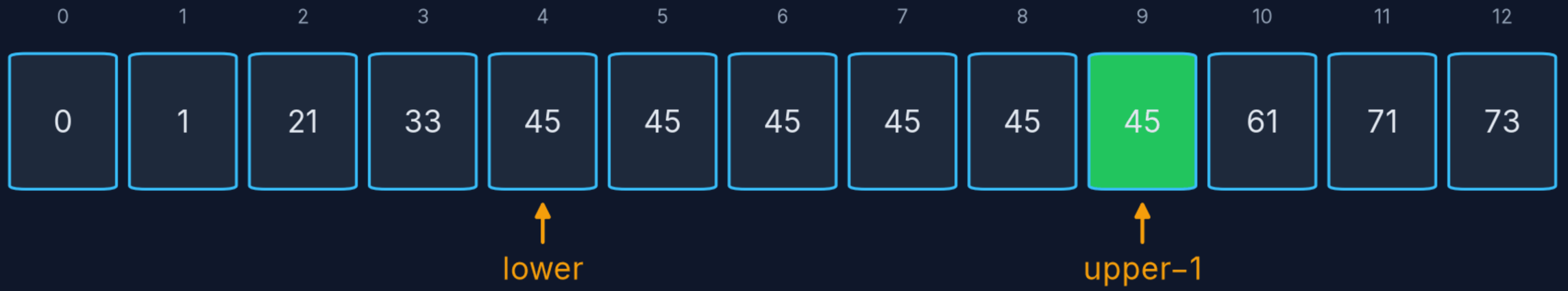
# Worked example




`first index with a[i] ≥ 45 → 4`

Lower bound of 45: 4

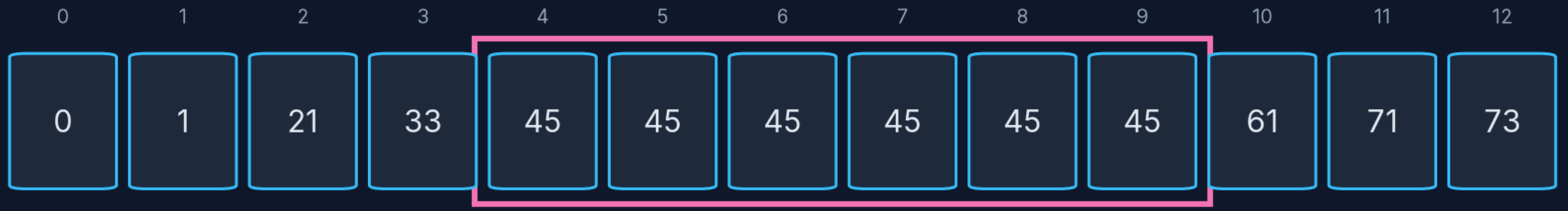
# Worked example




`first index with a[i] ≥ 46 → 10, minus 1 = 9`

Upper end: 9

# Worked example




```
answer [4, 9]
```

Answer: [4, 9]

# Java solution

```java
public class SearchForRange {
    public static int[] searchForRange(int[] a, int t) {
        int l = bound(a, t), r = bound(a, t + 1) - 1;
        return l <= r && l < a.length && a[l] == t ? new int[]{l, r} : new int[]{-1, -1};
    }

    private static int bound(int[] a, int t) {
        int lo = 0, hi = a.length;
        while (lo < hi) { int m = (lo + hi) >>> 1; if (a[m] < t) lo = m + 1; else hi = m; }
        return lo;
    }
}
```

# Complexity

- Time: O(log n)
- Space: O(1)

# Edge cases & pitfalls

- Target absent → [−1, −1]
- Half-open bounds
- Overflow-safe mid

# Interview tips & follow-ups

- Mention lower/upper bound terminology

# Recap

- Two binary searches: one biased left for the first occurrence, one biased right for the last.
- Complexity: O(log n)